# Vector-Based Models

# Vector Models & Text Preprocessing

- We want to study how Machine Learning is applied to language
- However, ML works on numbers, and language is made of text
- How to convert language to numbers, so that ML can be applied?
- **Text Preprocessing**!

# Basic Definitions

- Sentence: a sequence of words
- Ex: In English, a sentence typically begins with a capital letter and ends with punctuation (e.g. period, question mark)

Sentence: I love dogs.

# Tokens

- We may also consider a sentence to be a sequence of tokens
- Tokens can be words, but can also be punctuation, or sub-unit of words
- Note: it is common to use “tokens” in replace of “words”

Sentence: I love dogs.
Tokens: [I, love, dogs, .]

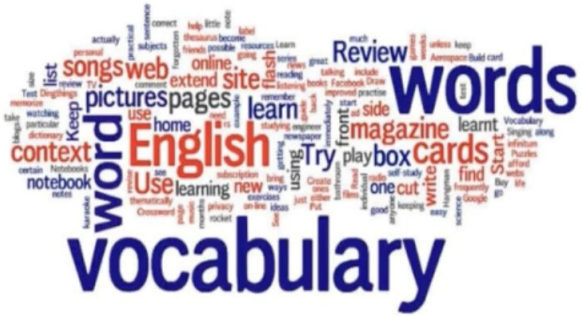
# Letters and Characters

- English is made up of 26 lettes (A-Z)
- A more general term: “character”
- Characters can represent punctuation and whitespace too (e.g. “\n”)
- In C++, Java, and others, the “character” is a fundamental data type
- Sometimes, NLP models will be build on words, but other times, they wiil be in terms of characters

```
#include<iostream>
using namespace std;
int main()
{
    char name[200][20];
    cin >> name[0] >> name[1] << na;
    cout << name[0] << '\t' << name[1];
    return 0;
}
```

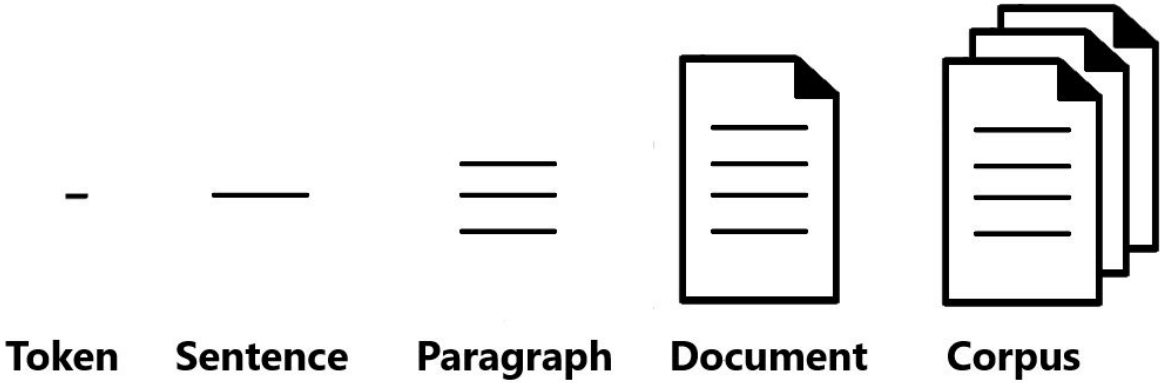
# Vocabulary

- Vocabulary: the set of “all” words
- In most cases, vocabulary won’t contain every possible word, but rather, just a “reasonable” subset of words
- How many to keep? 1000s? 10k’s? 100k’s? 1M’s?
- Depends on your application!
- Unless you are using a pretrained model (has been decided)



# Corpus

- “A large collection of writings of a specific kind or on a specific subject”
- Here, it will simply refer to our ML dataset



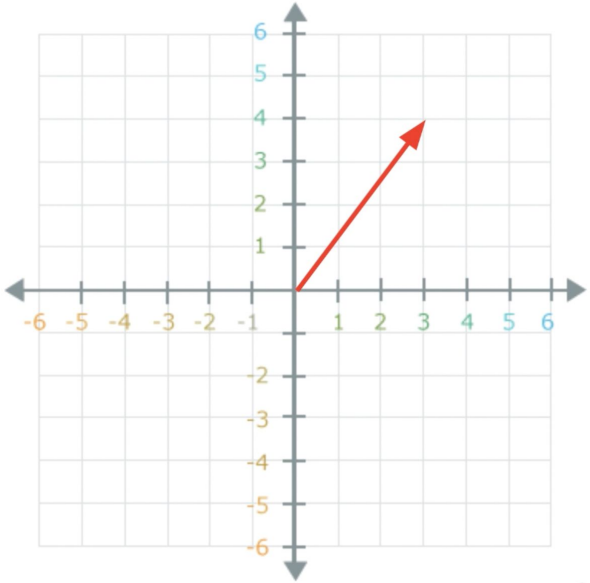
# N-Gram

- Refers to N consecutive items (e.g. words, subwords, characters)
- Single item: unigram
- Two items: bigram
- Three items: trigram

| Text | N-gram |
|---|---|
| Data | 1-gram |
| Great information | 2-gram |
| I am fine | 3-gram |
| Nice to meet you | 4-gram |

# What is a Vector?

- Vectors are how we will represent text numerically
- An array of scalars (e.g. (3, 4))
- Vectors can have 100s or 1000s of dimensions
- They are also the foundation of many ML techniques



# Why are Vectors Useful?

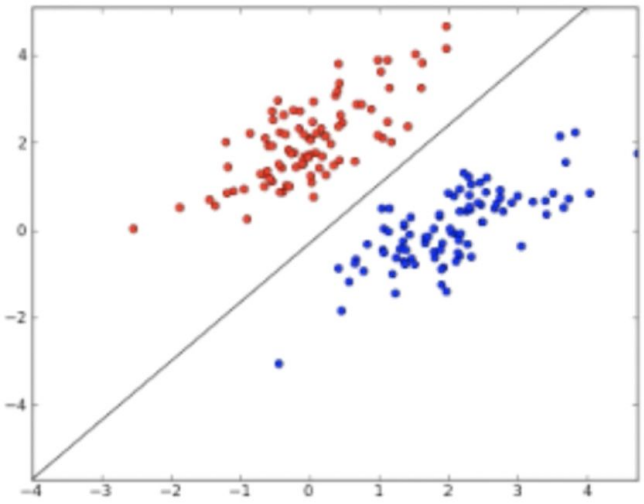
- To imagine why vectors are useful, think about  what you would do if you did not use vectors
- Task: *write a computer program that takes an email as input, and output whether or not the email is spam*
- What would you do?
  - Keywords
- If the email contains “Nigerian Prince”, “insurance”, “credit”, “make money”, etc. mark it as spam

# Why are Vectors Useful?

- What if you are interested in courses about computing credit score? You cannot filter on the word “credit”
- What if you taking a course on NLP, and the instructor uses the “Nigerian Prince” example? You cannot filter this phrase either
- What conditions makes an email safe? (NCHU?) Spammer can still send fake emails pretending to be from NCHU
- Overall problems: what to do if rules conflict?
- That’s why we need ML (data-driven automated decision-making)

# Spam Detection with Vectors

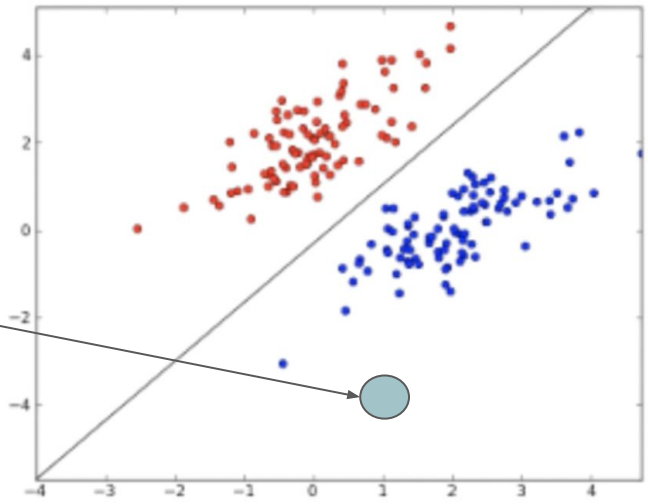
- Supose there is a way to map text into vectors, such that the vector for each class fall into their own clusters
- Then we can simply draw a line between the 2 clusters



# Spam Detection with Vectors

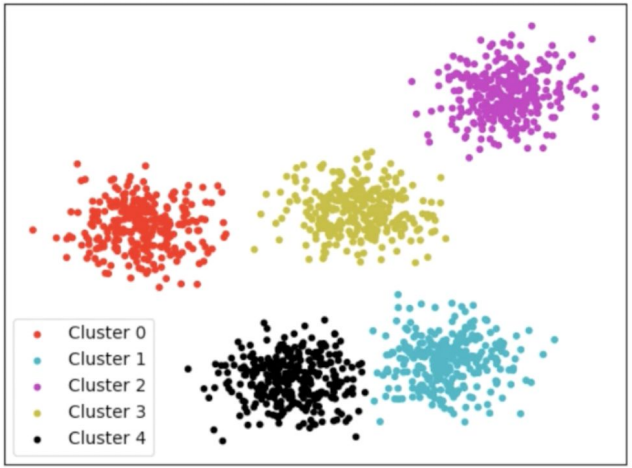
- Supose there is a way to map text into vectors, such that the vector for each class fall into their own clusters
- Then we can simply draw a line between the 2 clusters

- Suppose we have a new email whose spam status is unknown
- The question now is simple: which side of the line is it on?



# Organizing Documents

- Suppose a company has a large collection of document that needs to be organize, but there are way too many to read one-by-one
- We can automate this using **Clustering**
- But only if we convert the documents to vectors first

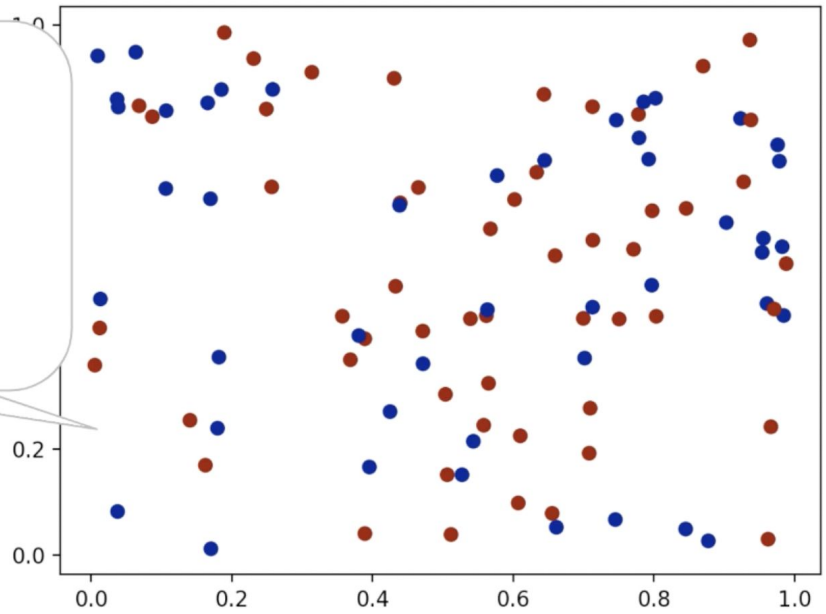


# Is it Possible?

- In reality, our vectors will not be so nicely separated

What we do **not** want.

Our objective will be to convert text into vectors *intelligently*.



# Bag of Words

- Text is sequential, the specific sequence of words gives the text meaning
- If we randomized the words in a sentence, we would change its meaning, or more likely, make it incomprehensible
- Despite this, many NLP approaches do not consider word order
- We will call this “bag of words” representations

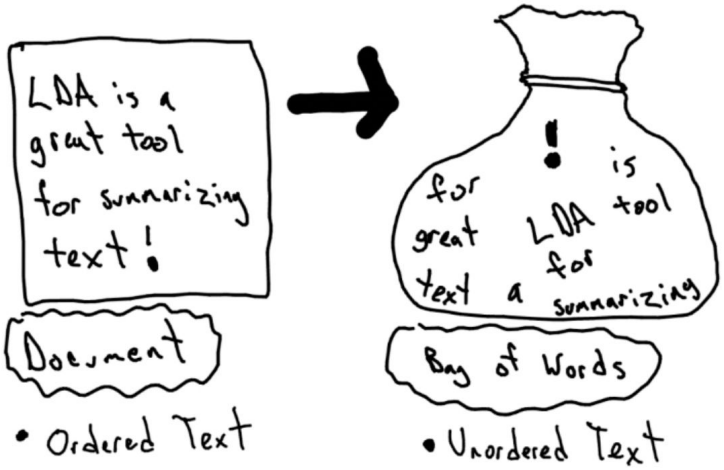
# Order Matters: Example

Dog Toy

Toy Dog

# Where is Bag of Words Used?

- Vector models and classic ML models use bag of words
- Probabalistic models and deep learning do not
- This is just a rough categorization: we can still find probabilistic and neural models that also use bag of words



# Count Vectorizer

- This method is simply called counting
- It is a “bag of words” approach
- Consider a simple classification task: discriminate between documents about biology and documents about physics

# Data Format

- Excel spreadsheet / CSV / Pandas dataframe (pre-requisite)

| Text (Inputs) | Labels (Targets) |
|---|---|
| Ad sales boost Time Warner profit\n\nQuarterly... | business |
| Dollar gains on Greenspan speech\n\nThe dollar... | politics |
| Yukos unit buyer faces loan claim\n\nThe owner... | entertainment |
| High fuel prices hit BA's profits\n\nBritish A... | finance |
| Pernod takeover talk lifts Domecq\n\nShares in... | business |

# Data Format

- Excel spreadsheet / CSV / Pandas dataframe

Our first goal is to convert the inputs to numerical values (vectors) for ML models

| Text (Inputs) | Labels (Targets) |
|---|---|
| Ad sales boost Time Warner profit\n\nQuarterly... | business |
| Dollar gains on Greenspan speech\n\nThe dollar... | politics |
| Yukos unit buyer faces loan claim\n\nThe owner... | entertainment |
| High fuel prices hit BA's profits\n\nBritish A... | finance |
| Pernod takeover talk lifts Domecq\n\nShares in... | business |

# Determining Vocabulary Size

- Let V = number of unique words in training corpus
- Each document will be converted into a vector of size V
- The vector components will contain the counts for each word

This means "and" appears in the document 25 times

An example vector

| | | | | | |
|---|---|---|---|---|---|
| 0 | 0 | 25 | ... | 0 | 3 |
| "a" | "aa" | "and" | ... | "zoo" | "zygote" |

# A Simple Example

- V (vocabulary size) = 6

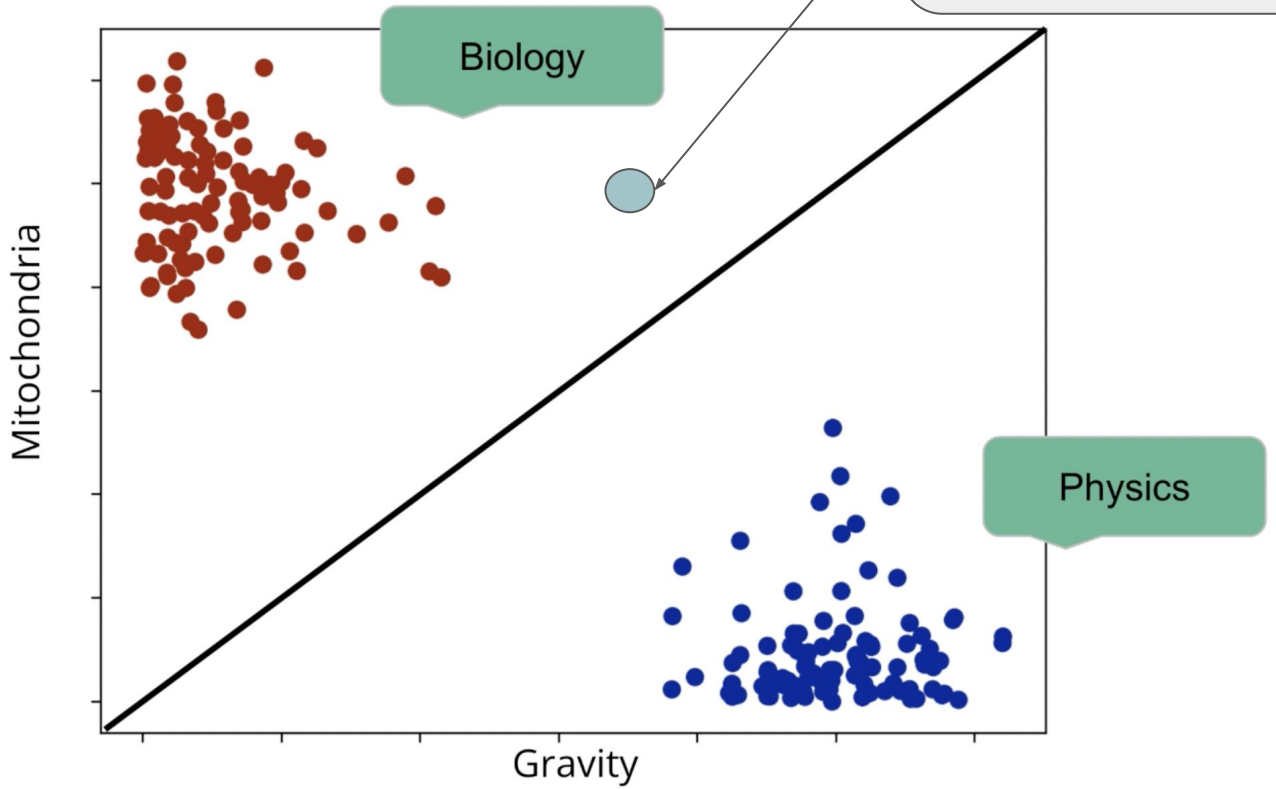
Each row is a vector of size 6

| Original Documents |
|---|
| I like eggs |
| I hate cats |
| I like eggs and I like cats |

| and | cats | eggs | hate | I | like |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 2 | 2 |

# How Does Counting Help?

- Given a new document, count up # Mitochondria, # Gravity, plot it on this graph, and determine which side of the line the vector falls on!



# CountVectorizer in Code (Scikit-Learn)

```
vectorizer = CountVectorizer()

vectorizer.fit(list_of_documents_train)

Xtrain = vectorizer.transform(list_of_documents_train)

# all in one step

Xtrain = vectorizer.fit_transform(list_of_documents_train)

Xtest = vectorizer.transform(list_of_documents_test)
```

# Normalization

- What if our corpus has very long and very short documents?
- Count vectors will have size disparities (more words = higher counts)

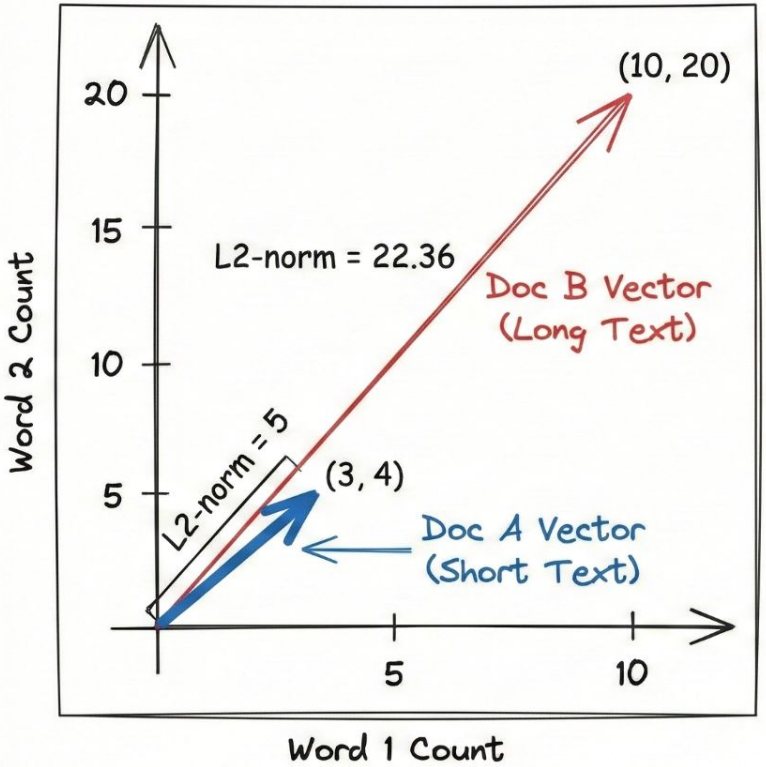
Method 1: Make the L2-norm 1

$$\hat{x} = \frac{x}{\|x\|_2}, \; where \; \|x\|_2 = \sqrt{\sum_{i=1}^{V} x_i^2}$$

Method 2: Divide by the sum (can think of each element as a probability)

$$\hat{x} = \frac{x}{\sum_{i=1}^{V} x_i}$$

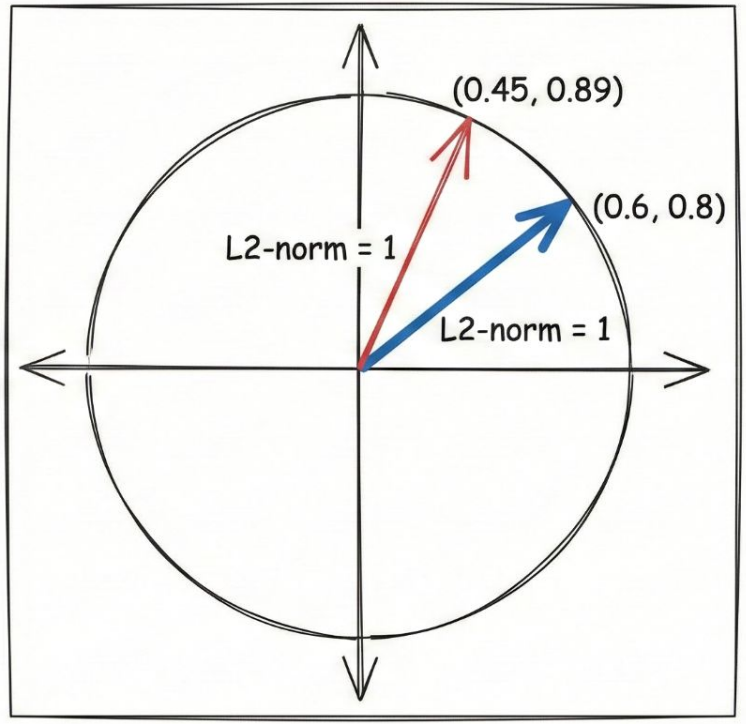
## Before Normalization (Length Varies)




Normalization Process:

Divide by its own L2-norm length

## After Normalization (Make L2-norm 1)




Result: All document vectors now have a uniform length of 1, enabling fair comparison (e.g., Cosine Similarity) regardless of original document length.

# Tokenization

- Tokenization: splitting strings into tokens
- Basic approach: s.split() → ["I", "like", "cats"]
- Better approach: Use NLP libraries (SpaCy, NLTK)

| Method | Output |
|---|---|
| s.split() | ["I", "like", "cats."] |
| NLTK word_tokenize() | ["I", "like", "cats", "."] |
| SpaCy Tokenizer | ["I", "like", "cats", "."] |

# Tokenization: Words-Based VS. Character-Based

Cons of word-based tokenization:

- Up to 1 million words (hence, 1 million dimensions)
- Must have # outputs equal to vocabulary size (1 million dimension probability distribution)
- This corresponds to a very large weight matrix

Pros of word-based tokenization:

- Words have meaning / contain lots of information

# Tokenization: Words-Based VS. Character-Based

Cons of charater-based tokenization:

- Characters do not contain lots of information

Pros of character-based tokenization:

- In English, there are only 26 letters, + some characters for white space and punctuation
- Thus, the “vocabulary” size is small, easy to represent in a computer

# Subword-Based Tokenization

- A middle ground between word-based and character-based
- Ex: “walking” → “walk” + “ing”
- “Walk” is closely related to “walking” - we’d like them to have some shared representation in our model (via the token “walk”)
- “ing” is a meaningful token - it has the same meaning whether it’s applied to “walking”, “eating”, etc.

# Subword-Based Tokenization

- What if we didn’t split “walking” into “walk” + “ing”?
- Each vector component (count) is separate, so “walk” is no closer to “walking” than it is to “tree”
- We can only hope our model learn the similarity through the data
- Do we want our model to learn “walk”, “walks”, “walking”, “walked”, etc. independently? Or should we connect them via a shared representation?

# In Scikit-Learn

Word-based tokenization

```
CountVectorizer(analyzer=”word”)
```

Character-based tokenization

```
CountVectorizer(analyzer=”char”)
```

# Punctuation

- Punctuation may be important for downstream NLP tasks
- Ex: “I hate cats.” vs. “I hate cats?”
- Tokenizing punctuation: “I hate cats?” → [“I”, “hate”, “cats”, “?”]
- Should you use it? Thats dependent on your experimental results
  - Semtiment analysis → Yes (affect meaning)
  - Topic modeling → No (add noise)
- Note: SKLearn CountVectorizer ignores punctuation

# Casing

- Consider sentiment analysis or spam detection
- Does “cat” have the same meaning as “Cat”?
- Simply call `string.lower()` in Python
- With SKLearn: `CountVectorizer(lowercase=True)`

# Accents

- Less common in English
- “Naive” and “Náive” are both correct
- Map the character yourself, or use `CountVectorizer(strip_accents=True)`

# Stopwords

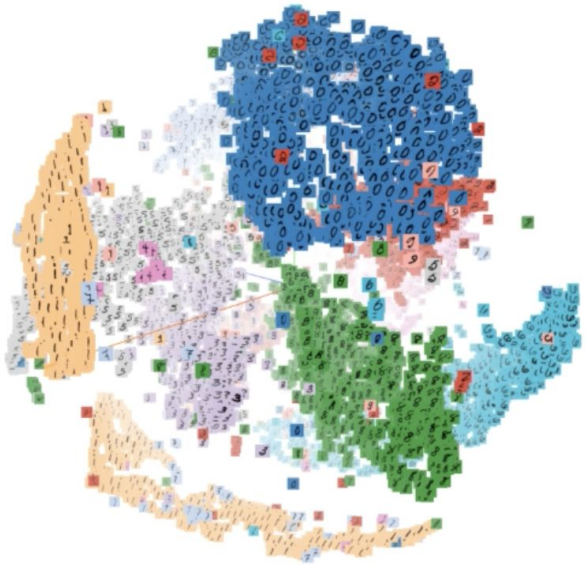
- What we know so far: how to build vector using counting
- Tokenization → Count the tokens → Put them into a vector

# Why do We Need Stopwords?

- When we perform counting procedure, what happen with very common words? such as “and”, “the”, “it”, “is”, etc.
- Are these words useful?
- Any text will contain these words

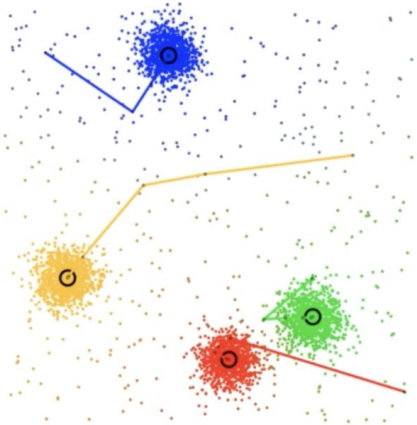
# Dimensionality

- High dimensionality is bad
- More dimensions → more computation
- Could it be beneficial to simply not include stopwords?



# Distance Consideration

- Recall that our understanding of feature vectors involves theirs distance to each other
- If we count all the “and”s, “the”s, etc. the vectors which will be close will simply by the ones with similar numbers of these words
- The might overshadow more important words, like “mitochondria” or “voltage”
- Thus, **stopwords** are the words we wish to ignore



# Stopwords in CountVectorizer

```
CountVectorizer(stop_words=”english”)

CountVectorizer(stop_words=list_of_user_defined_words)

CountVectorizer(stop_words=None) # default
```

# NLTK Stopwords

```
import nltk

nltk.download('stopwords')

from nltk.corpus import stopwords


stopwords.words('english')

stopwords.words('german')
```

中文需使用 jieba 或 CKIP 進行 tokenization，並使用自定義的停用詞表

# Stemming & Lemmatization

- With basic word tokenization, each variation of a word will have its own vector component: “walk”, “walking”, “walks”, “walked”, …
- “Walk” is no closer to “walking” than it is to “cartwheel” (unless similarity can be learned from the data)
- This also leads to high-dimensional vectors
- Practical issue: imagine we’re building a search engine, and search for “running” (what about “ran” and “run”? How to match them?)
- Solution: convert words to their root word
- 2 methods: stemming and lemmatization

# Stemming & Lemmatization

- Stemming is very **crude** - it just cut off the end of the word
- The result is not necessarily a real word
- Lemmatization is more **sophisticated** - it uses actual rules of language
- The true root word will be returned

# Stemming

- Based on single heuristics
- Ex: ends with “sses” → remove “es”
- Ex: “Bosses” → “Boss”
- Ex: “Replacement” → “Replac”
- Multiple stemming algorithms
  - Porter Stemmer
  - Lancaster Stemmer
  - Snowball Stemmer

```
from nltk.stem import PorterStemmer

porter = PorterStemmer()

porter.stem(“walking”) # returns “walk”
```

# Lemmatization

- Think of it as a lookup table
- Stemming: “Better” → “Better”
- Lemmatization: “Better” → “Good”
- Note: “Was” is the past-tense of “Is”, both are derivatives of “Be”
- Stemming: “Was” → “Wa”
- Lemmatization: “Was” / “Is” → “Be”
- Stemming: “Mice” → “Mice”
- Lemmatization: “Mice” → “Mouse”

# How to Use Lemmatization

- Appear in NLTK, spaCy, and others

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
nltk.download("wordnet") # only need to do once

lemmatizer = WordNetLemmatizer()
lemmatizer.lemmatize("mice") # returns 'mouse'

lemmatizer.lemmatize("going") # returns 'going'
lemmatizer.lemmatize("going", pos=wordnet.VERB) # returns 'go'
```

# Why Does the Part-of-Speech Matter?

"Donal Trump has a devoted following"

Noun

"The cat was following the bird as it flew by"

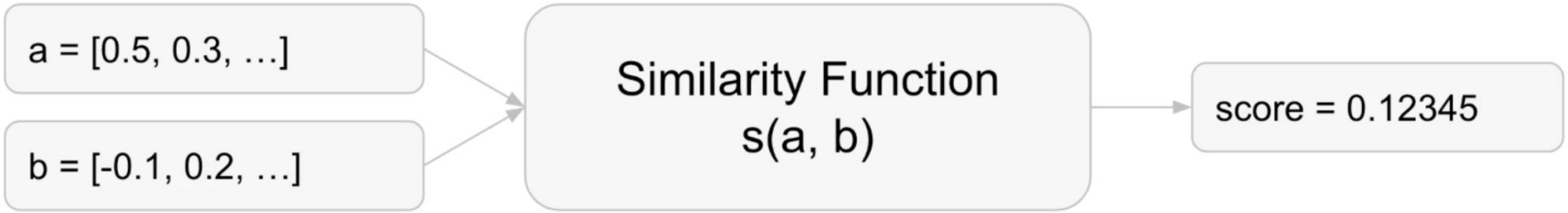
Verb

# Why Stemming & Lemmatization Still Matter in NLP?

- **Misconception**: Are These Techniques Outdated?
  - Some believe that NLP today is all about “throwing everything at deep learning”—but this approach is inefficient and lacks interpretability.

- **Reality**: Stemming & Lemmatization Are Still Essential!
  - These techniques remain crucial preprocessing steps in various NLP applications, such as:
  - Search Engines / Document Retrieval (Google, Bing)
  - Online Advertising (Google Ads, Meta Ads)
  - Social Media Tagging (Twitter, Instagram)
- **Why?**
  - They reduce word variations, improve text normalization, and enhance computational efficiency—helping both traditional NLP models and modern AI systems.

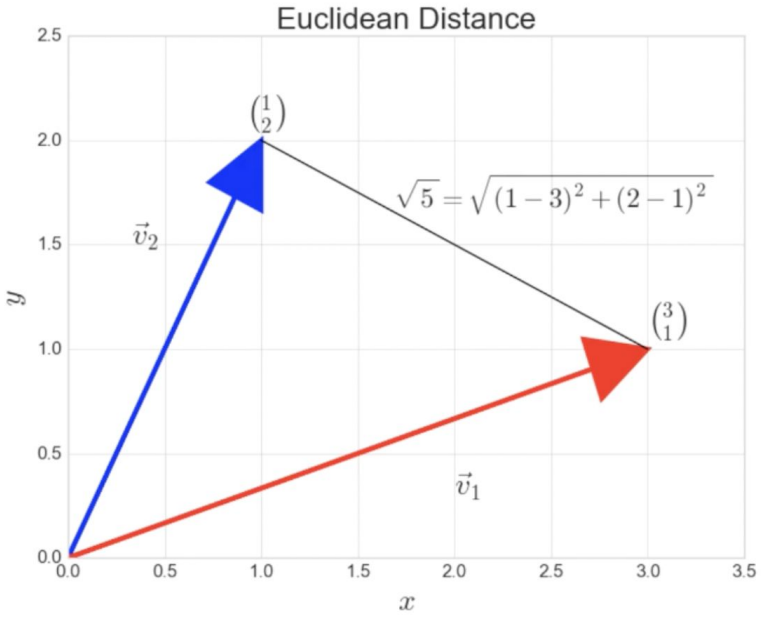
# Vector Similarity




- How is it useful? Given some document, find the “most similar” document
- E.g. Researchers looking for papers on a subject
- If we have word vectors: we may find “king” and “queen” are similar
- “Car”, “vehicle”, “automobile” → similar
- “Car”, “giraff” → dismilar

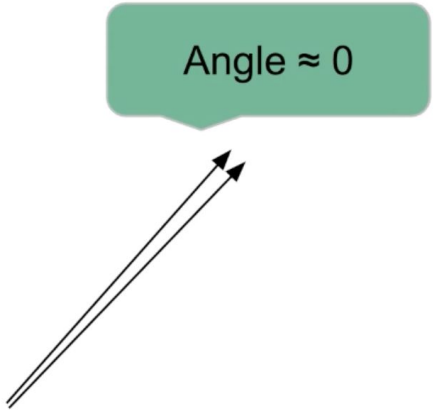
# Euclidean Distance

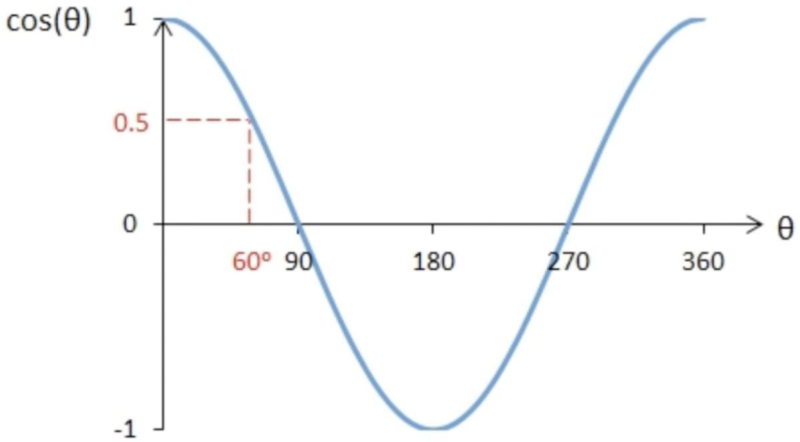
- In a Euclidean space, we can calculate the straight-line distance between two document vectors.
- While distance can measure similarity, it is sensitive to document length.
- For best results, this is often used with normalized vectors.



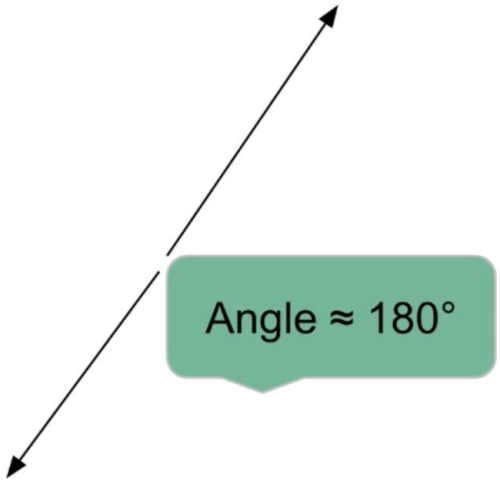

$$\|x - y\|_2 = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \ldots + (x_D - y_D)^2}$$

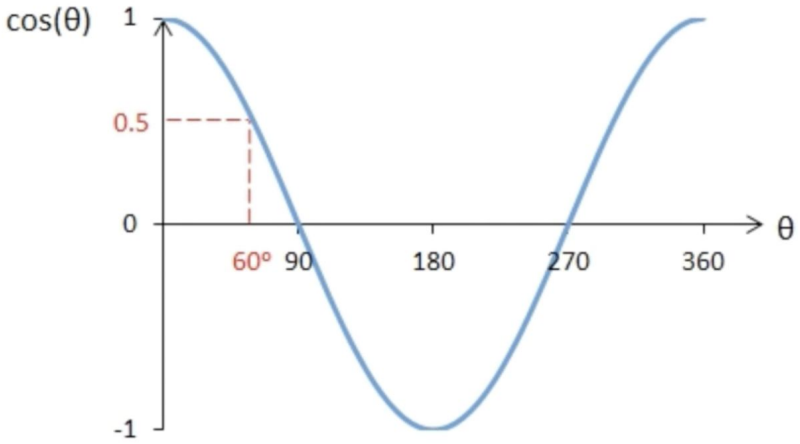
# Cosine Similarity



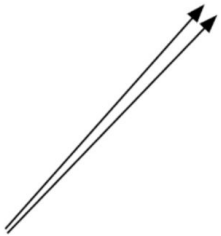
# Cosine Similarity



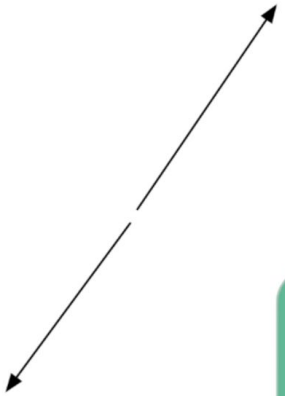



# Cosine Distance

$$Cosine\ Distance = 1 - Cosine\ Similarity$$

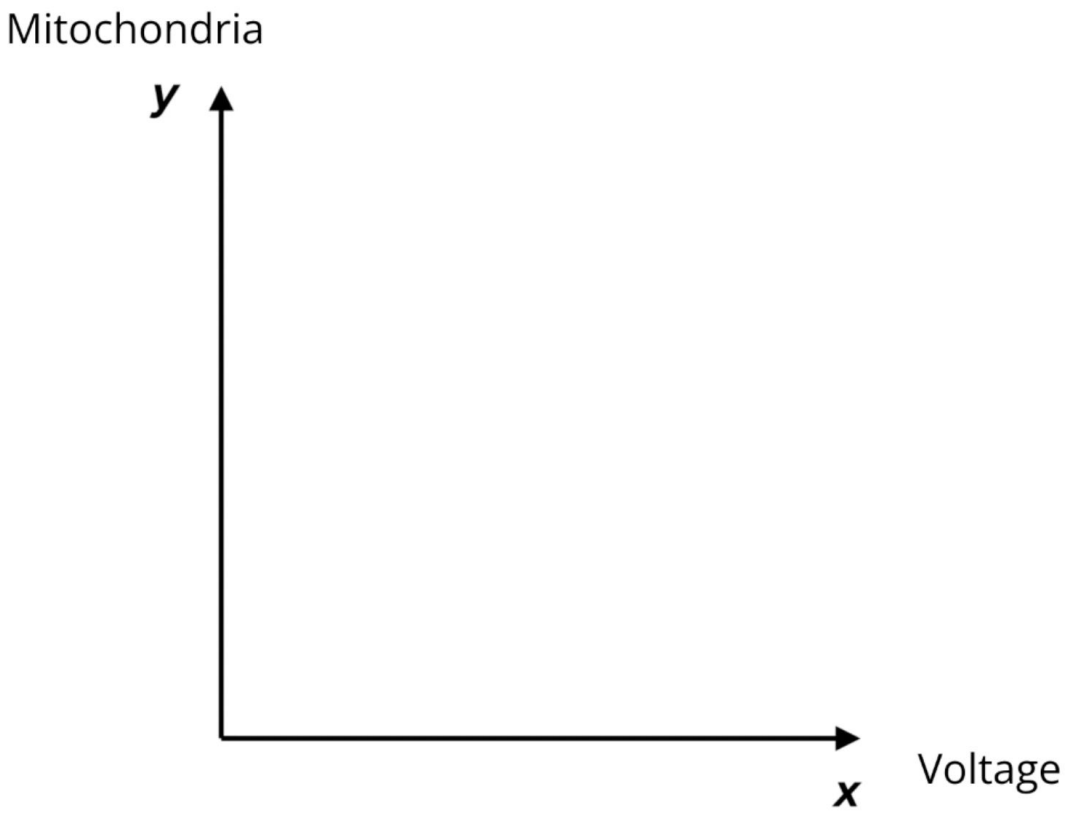


dist = 1 - sim
dist = 1 - 1
dist = 0

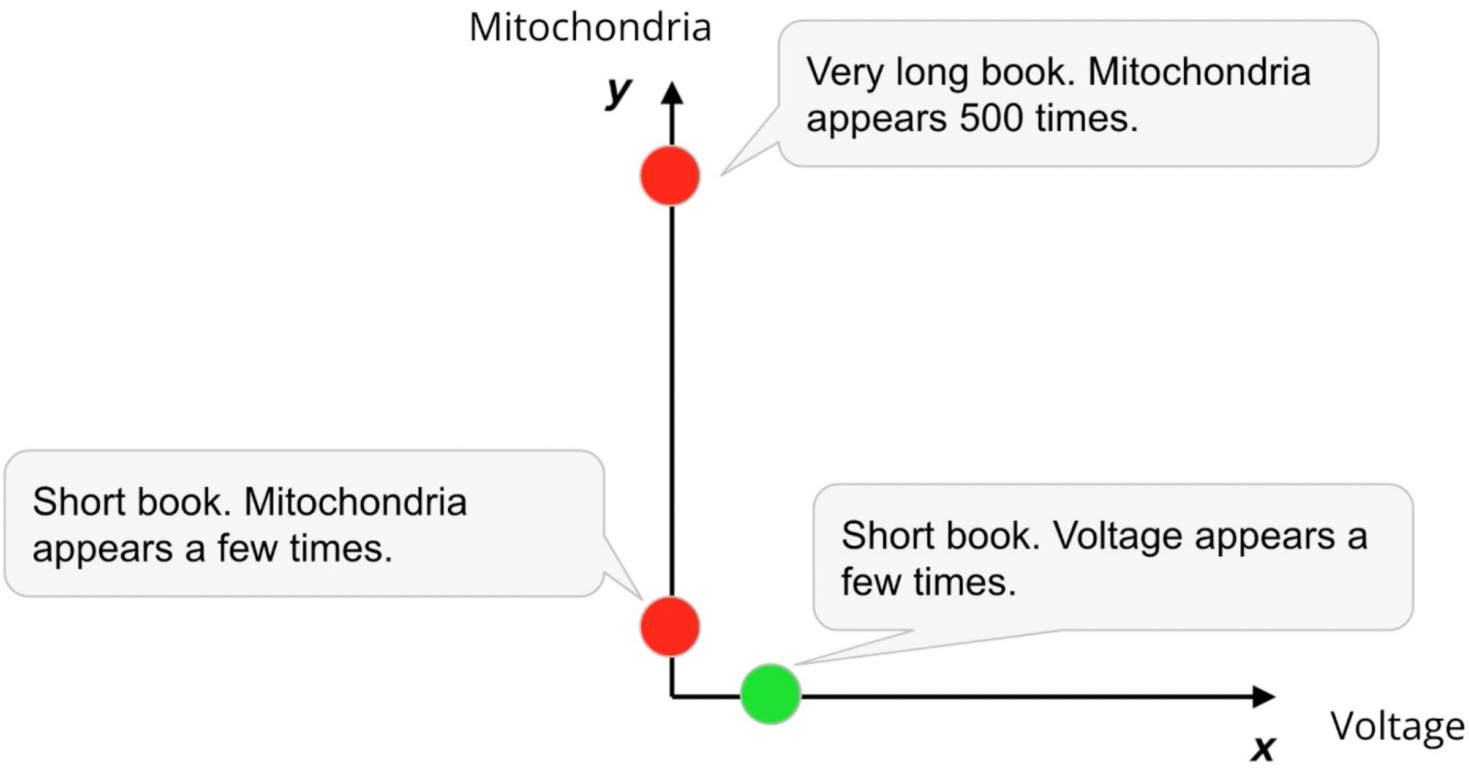


dist = 1 - sim
dist = 1 - (-1)
dist = 2

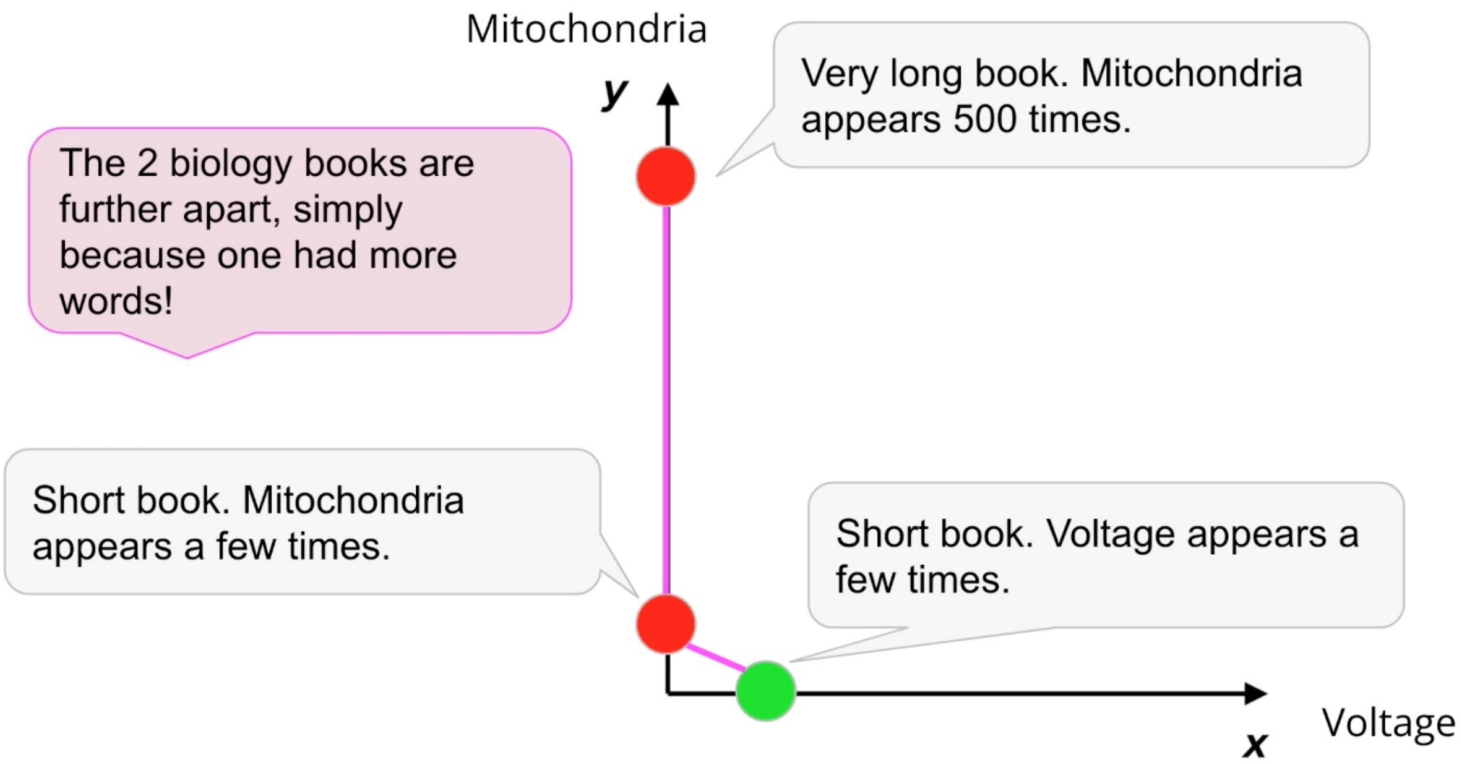
# Which One Should We Use?



# Which One Should We Use?



# Which One Should We Use?



# TF-IDF

- Improve the count vectorizer
- TF-IDF is popular for document retrieval and text mining
- Why we need it and how it works

# What’s Wrong with the Count Vectorizer?

- Why don’t we want to keep stopwords?
- They’re unlikely to be helpful for NLP tasks
- If you’re building a search engine:
  - Probably every document contain these words
- If you’re doing spam detection or sentiment analysis:
  - Probably every document contain these words

# Stopwords

- How do we know our list of stopwords is correct?
- They may be application-specific
- “Mitochondria” might be useful to differentiate biology and physics
- But what if all documents are about biology
- Can we automatically determine which words are important, and which to ignore?

# The Main Idea Behind TF-IDF

- Words that we want to ignore appear in many different documents
- They won’t help us differentiate between documents
- We want to somehow scale down these word counts

# TF-IDF: Intuitive Idea

- "**T**erm **F**requency - **I**nverse **D**ocument **F**requency"

$$TF - IDF \approx \frac{Term\ Frequency}{Document\ Frequency}$$

# TF-IDF Formula

The term count depends on which term t we are counting, and which document d we are looking at

$$tfidf(t,d) = tf(t,d) \times idf(t)$$

The document count only depends on the term t. It is found by summing over **all** documents.

# TF-IDF

- Term Frequency
  - This is what we get with the CountVectorizer (after calling “transform”)

$$tf(t,d) = \#\ of\ times\ t\ appears\ in\ d$$

- Inverse Document Frequency
  - Let N(t) be the number of documents term t appears in and N be the number of documents

$$idf(t) = \log \frac{N}{N(t)}$$

# Why Take the Log?

- The log is a monotonic function: if N/N(t) gets larger, so too will its log
- i.e. TF-IDF goes down as t appears in more documents
- The log function squashes its argument
- Suppose N(t) = 1, and N = $10^6$, if we didn't take the log, TF-IDF would increase by 1 million (relative to TF)
- With logging, this goes down to 13.8 (better for ML)
- Deeper reasons: information theory

$$idf(t) = \log \frac{N}{N(t)}$$

# How to Use TF-IDF in Python

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer()
Xtrain = tfidf.fit_transform(train_texts)
Xtest = tfidf.transform(test_texts)

# note: arguments exist for stopwords, tokenizer, strip
accents, etc.
```

# Term Frequency Variations

- Binary (1 if word appears, 0 otherwise)
- Normalize the count

$$tf(t,d) = \frac{count(t,d)}{\sum_{t' \in terms(d)} count(t',d)}$$

- Take the log

$$tf(t,d) = \log(1 + count(t,d))$$

# Inverse Document Frequency Variations

- Smooth IDF

$$idf(t) = \log \frac{N}{N(t)+1} + 1$$

- IDF Max

$$idf(t) = \log \frac{\max_{t' \in terms(d)} N(t')}{N(t)}$$

- Probabilistic IDF

$$idf(t) = \log \frac{N - N(t)}{N(t)}$$

# Normalizing TF-IDF

- Unlike CountVectorizer, TfidfVectorizer supports it
- TfidfVectorizer(norm='l2') (or 'l1'; L2 is the default)

$$\hat{tfidf}(t,d) = \frac{tfidf(t,d)}{\|tfidf(\cdot,d)\|_p}, \; where\ p = 1\ or\ 2$$

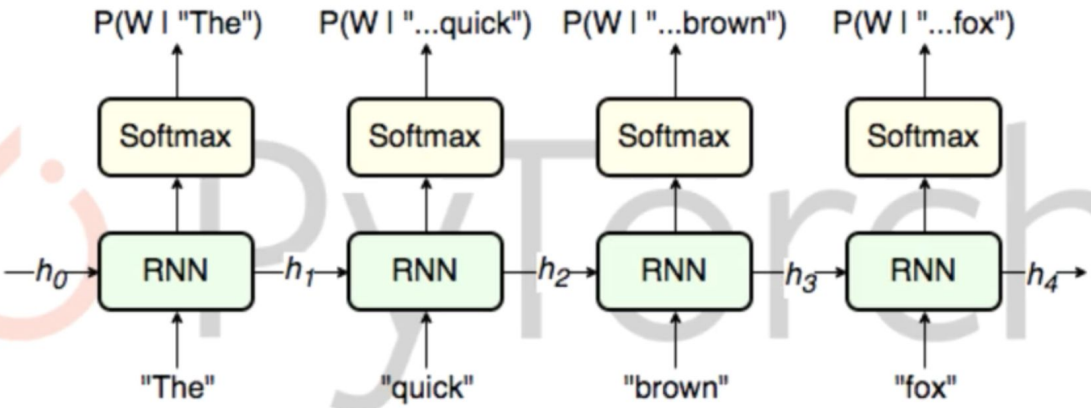
# Exercise - TF-IDF Recommendation system

- Data: movie info (keywords, genre. title, synopsis, tagline, production, …)
- Key step: how to combine movie data into a single string?
  - Recall: TfidfVectorizer expects one string per “Document”
- Transform the strings using TF-IDF
- Assume the query is always an existing movie in the database
- E.g. query = “Scream 3”, then recommend other movies based on this
- Get TF-IDF representation of Scream 3
- Compute similarity between Scream 3 and all other vectors
- Sort by similarity
- Print out the top 5 closest movies
- Try movies from other genres

# Neural Word Embeddings

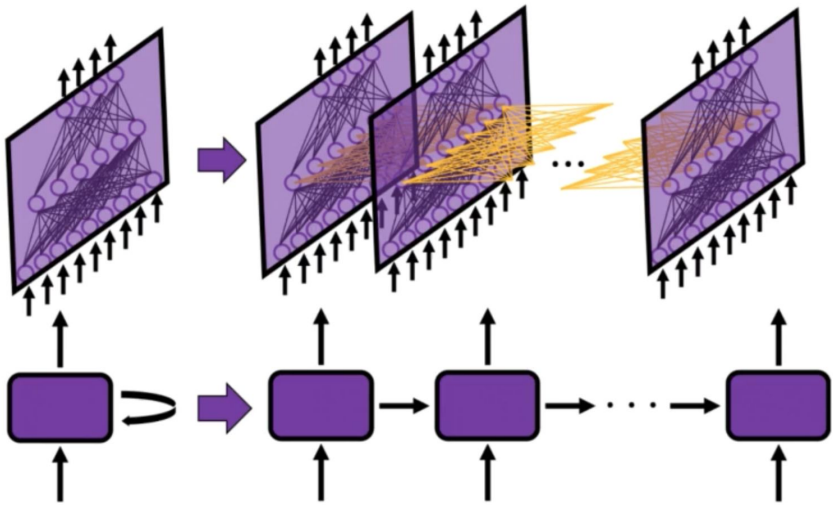
- Previously, we saw how to convert whole documents into vectors
- Neural word embeddings convert single word into vectors
- Thus, a document becomes a sequence of vectors
- More information than “bag of words”

Just think of “word embedding” the same as “word vector”



# Sequence of Vectors

- Specialized models are designed to handle sequences, including:
  - CNNs
  - RNNs
  - Transformers
- Cutting-edge applications include:
  - Language translation
  - Question answering
  - Chatbots
  - Speech-to-text and text-to-speech
  - Biological sequence analysis



# Embedding Methods

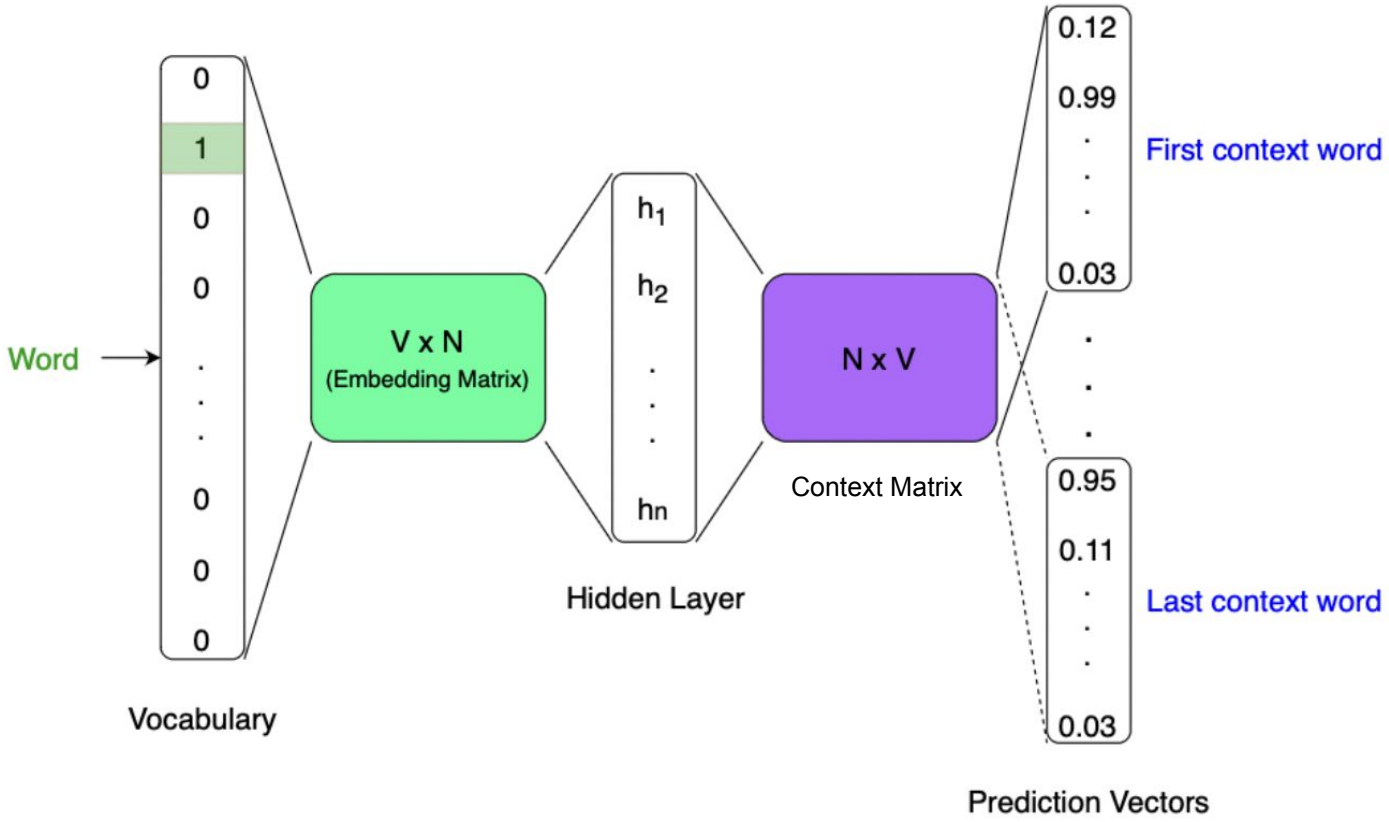
- Word2Vec (Google) – Predicts word context using neural networks.
- GloVe (Stanford) – Uses word co-occurrence statistics for embeddings.
- BERT – Contextual embeddings with bidirectional transformer models.

# Word2Vec

- Word2Vec learns word representations based on surrounding words.
- Uses a simple neural network (embedding matrix) with only one hidden layer.
- Two training methods:
  - Continuous Bag of Words (CBOW): Predicts a target word from surrounding words.
  - Skipgram: Predicts surrounding words given a target word.
- Once trained, only the embedding matrix is used.

https://towardsdatascience.com/implementing-word2vec-in-pytorch-from-the-ground-up-c7fe5bf99889/

# Word2Vec Architecture (Skipgram)



# What is the Embedding Matrix?

- A  V x N matrix storing word embeddings.
  - V = Vocabulary size
  - N = Embedding dimension
- Each row represents a word vector.
- Words with similar meanings have similar vectors.

| Word | Learned Word Vector (Embedding Matrix) |
|---|---|
| king | [0.27, -0.63, 0.89, ...] |
| queen | [0.31, -0.60, 0.85, ...] |
| apple | [0.12, -0.78, 0.35, ...] |

# Pros & Cons of Word2Vec

Strengths:

- Captures semantic meaning and word relationships effectively.
- Generates dense, low-dimensional vectors (solves the curse of dimensionality).
- Highly efficient and scalable for training on massive datasets.
- Provides easily transferable pretrained embeddings for downstream NLP tasks.

Limitations:

- Cannot handle out-of-vocabulary (OOV) words.
- Ignores subword information and morphology (e.g., apple vs. apples, solved by FastText).
- Does not understand context variations and polysemy (e.g., the word "bank", solved by BERT).
- Limited by a fixed context window, missing long-distance semantic dependencies.

# GloVe

- A word embedding model developed by Stanford University.
- Uses co-occurrence statistics to learn word relationships.
- Captures global word meaning by analyzing entire corpora.
- Unlike Word2Vec, it does not use a neural network.

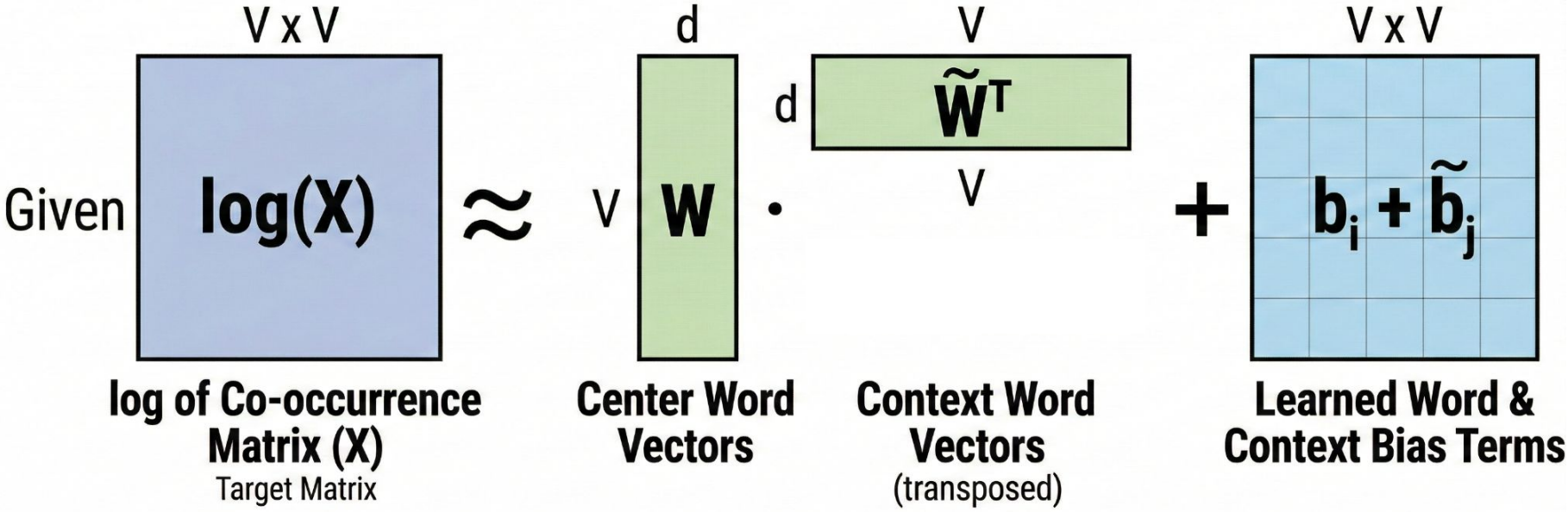
# Learning Word Embeddings with GloVe

- Step 1: Build a word co-occurrence matrix from a large corpus.
- Step 2: Apply matrix factorization to generate dense word vectors.

Example of Co-Occurrence:

- If "ice" appears 20 times with "cold" but only 1 time with "steam", GloVe assigns a higher similarity between "ice" and "cold".
- Context words like "winter" or "freeze" will also be closer to "ice" in vector space.
- Word relationships emerge from co-occurrence patterns.

# Matrix Factorization



# Pros & Cons of GloVe

Strengths:

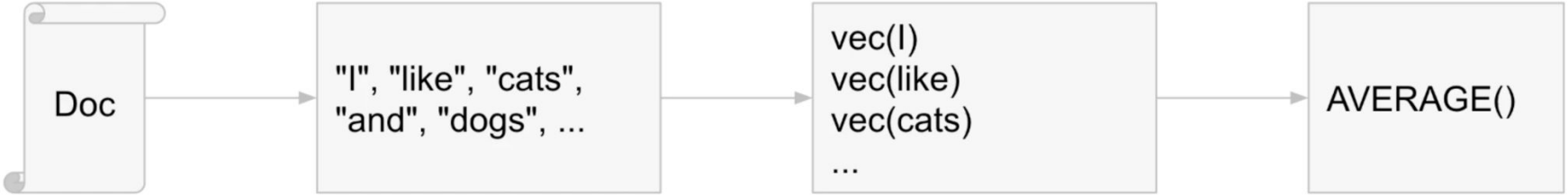
- Similar to Word2Vec
- Captures both global and local word relationships.

Limitations:

- Requires pre-computing a co-occurrence matrix (memory-intensive).
- Cannot learn new words dynamically after training.
- Does not understand context variations (solved by BERT).

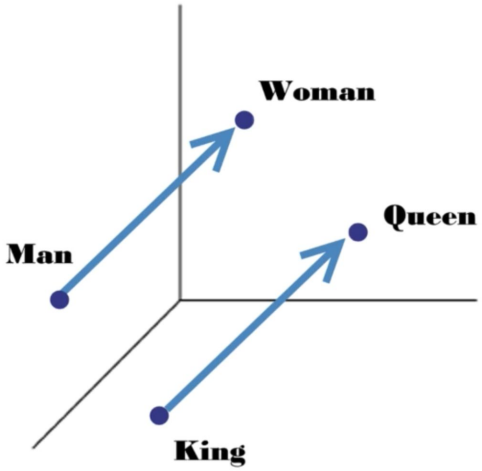
# What can We Do with Word Vectors?

- Can convert a document into a vector (but not sparse like counting/TFIDF)
- Embeddings are dense and low dimensional (20, 50, 100, 300, … << V)



# Word Analogies

- How do we evaluate our embedding model?
- We can do arithmetic on vectors (+ and -)
- King : Man :: ??? : Woman
- Answer: Queen
- In math: King - Man ≈ Queen - Woman
- In code: x = King - Man + Woman
  - Find the closest word vector to x
  - The result will be Queen



# Word Analogies

- France : Paris :: Italy : Rome
- Japan : Japanese :: China : Chinese
- Miami : Florida :: Dallas : Texas
- December : November :: July : June
- Man : Woman :: He : She

# Vector Models & Text Preprocessing Summary

- Connection between text (strings) and numbers, i.e., turning text into vectors
  - Counting
  - TF-IDF
  - Neural word embeddings (word2vec, GloVe)
- Text preprocessing:
  - Tokenization
  - Bag of words
  - Stopwords
  - Stemming and lemmatization

# NLP in Other Languages

- Everything in this course can be applied to any language
- Reminder: many NLP techniques are applied in bioinformatics / genomics
- Strings of DNA / RNA / etc.

# Steps of a Typical NLP Analysis

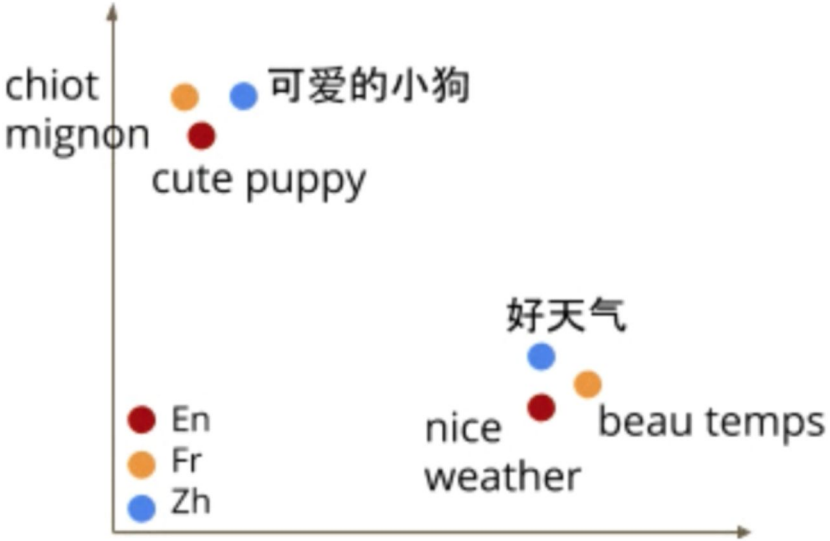
- Get the text (strings)
- Tokenize the text
- Stopwords, stemming / lemmatization
- Convert text into count vectors / TF-IDF
- Do ML task (recommend, detect spam, summarize, topic model)

# When Working with a New Language …

- Some questions to ask
  - What is a “word”?
  - What is a “sentence”? How does punctuation work?
  - Boundaries between sentences?
  - Boundaries between words?
  - Stopwords?
- Find a tokenizer built by others or build one yourself
- Learn the language!

# Multilingual Models

- Idea: try to make embedding consistent across languages



# Multilingual Models

- Challenge: Different languages have unique grammar, word order, and cultural nuances.
- Methods:
  - **Supervised Alignment**: Uses parallel corpora (e.g., translations) to align embeddings.
  - **Unsupervised Alignment**: Learns alignment without direct translations, relying on statistical patterns.
  - **Joint Training**: Trains embeddings on multiple languages at the same time.